



BLG308 Yazılım Mühendisliği Dönem Projesi

Ders Kodu ve Adı: BLG308 Yazılım Mühendisliği

Proje Başlığı: React, Node.js, Express.js ve MySQL Kullanılarak Rol Tabanlı Görev

Yönetim Sistemi Geliştirilmesi

Öğrenci Adı Soyadı: Dila Nur DAN

Öğrenci Numarası: 231201042

Öğretim Üyesi: Dr. Öğr. Üyesi Fatima Bhutta

Bölüm: Bilgisayar Mühendisliği

Dönem: 2025–2026 Bahar Dönemi

Teslim Tarihi: Haziran 2026

React, Node.js, Express.js, MySQL ve JWT Tabanlı Rol Tabanlı Görev Yönetim Sistemi Geliştirilmesi

Özet

Bu çalışmada React, Node.js, Express.js, Sequelize ORM ve MySQL teknolojileri kullanılarak web tabanlı bir görev yönetim sistemi geliştirilmiştir. Geliştirilen sistem, kullanıcıların güvenli şekilde kayıt olabildiğini, giriş yapabildiğini ve görevlerini yönetebildiğini sağlamaktadır. Ayrıca sistem içerisinde Admin, Manager ve User olmak üzere üç farklı kullanıcı rolü tanımlanmış ve rol tabanlı erişim kontrolü (RBAC) uygulanmıştır.

Frontend katmanında React ve Vite kullanılarak dinamik, bileşen tabanlı ve kullanıcı dostu bir arayüz geliştirilmiştir. Backend katmanında Express.js ile RESTful API mimarisi uygulanmış, Sequelize ORM kullanılarak MySQL veritabanı işlemleri yönetilmiştir. Kullanıcı kimlik doğrulama işlemleri JSON Web Token (JWT) ile gerçekleştirilmiş, kullanıcı şifreleri bcrypt algoritması ile hashlenerek güvenlik artırılmıştır.

Bu çalışmanın temel amacı, modern yazılım mühendisliği prensiplerine uygun şekilde güvenli, sürdürülebilir, modüler ve ölçeklenebilir bir görev yönetim sistemi geliştirmektir. Ayrıca sistem, istemci-sunucu mimarisi, middleware tabanlı güvenlik kontrolü ve rol tabanlı yetkilendirme gibi güncel web teknolojilerini içermektedir.

Anahtar Kelimeler: React.js, Node.js, Express.js, JWT, RBAC, Sequelize ORM, MySQL, Görev Yönetim Sistemi

1. Giriş

Görev yönetim sistemleri, bireysel kullanıcıların ve kurumların iş süreçlerini düzenli şekilde takip edebilmesini sağlayan önemli yazılım çözümleridir. Günümüzde dijitalleşmenin artmasıyla birlikte web tabanlı görev yönetim uygulamaları hem bireysel hem de kurumsal kullanım açısından yaygın hale gelmiştir.

Modern web uygulamalarında kullanıcı deneyimi, güvenlik, performans ve sürdürülebilirlik önemli kriterler arasında yer almaktadır. Bu nedenle geliştirilen sistemlerin yalnızca işlevsel olması yeterli değildir; aynı zamanda güvenli, ölçeklenebilir ve modüler mimariye sahip olması gerekmektedir.

Bu projede React tabanlı frontend yapısı ile Node.js ve Express.js tabanlı backend sistemi entegre edilerek tam kapsamlı bir görev yönetim sistemi geliştirilmiştir. Sistem, kullanıcıların görev oluşturmaya, görevlerini görüntülemesine, güncellemesine ve silmesine olanak

sağlamaktadır. Ayrıca Admin ve Manager rolleri sayesinde sistem yönetsel açıdan da kontrol edilebilmektedir.

REST mimarisi üzerine kurulan sistem, istemci ve sunucu katmanlarının birbirinden bağımsız şekilde çalışmasına olanak sağlamaktadır. Bu yaklaşım, yazılım bakımını kolaylaştırmakta ve sistemin gelecekte genişletilebilir olmasını desteklemektedir.

2. Literatür Taraması ve İlgili Çalışmalar

Web tabanlı yazılım sistemlerinin geliştirilmesinde REST mimarisi günümüzde yaygın olarak kullanılmaktadır. REST yaklaşımı ilk olarak Fielding (2000) tarafından tanımlanmış ve istemci-sunucu iletişimde esneklik, ölçeklenebilirlik ve bağımsızlık sağlaması nedeniyle modern web servislerinde standart hale gelmiştir.

Kimlik doğrulama sistemlerinde JSON Web Token (JWT) kullanımı son yıllarda önemli ölçüde yaygınlaşmıştır. JWT yapısı, sunucu tarafında oturum bilgisini saklamadan kullanıcı doğrulama işlemlerinin gerçekleştirilebilmesini sağlamaktadır. Jones, Bradley ve Sakimura (2015), JWT kullanımının özellikle stateless sistemlerde performans avantajı sunduğunu belirtmektedir.

Benzer görev yönetim sistemleri üzerine yapılan çalışmalarda kullanıcı erişim kontrolü ve veri güvenliği önemli araştırma konuları arasında yer almaktadır. Zhang, Chen ve Li (2019) tarafından gerçekleştirilen çalışmada web tabanlı görev yönetim sistemlerinin özellikle kullanıcı yetkilendirme süreçlerinde güvenlik açıkları oluşturabileceği belirtilmiştir.

Stallings (2017), güvenli yazılım sistemlerinde parola güvenliğinin kritik öneme sahip olduğunu vurgulamaktadır. Bu nedenle modern sistemlerde kullanıcı şifrelerinin hash algoritmaları ile saklanması önerilmektedir. Bu projede bcrypt algoritması kullanılarak kullanıcı şifreleri güvenli hale getirilmiştir.

Ayrıca literatürde rol tabanlı erişim kontrolü (RBAC) sistemlerinin büyük ölçekli uygulamalarda yönetim kolaylığı sağladığı belirtilmektedir. Sandhu ve arkadaşları (1996), RBAC modelinin kullanıcı yetkilerini merkezi şekilde yönetebilmesi nedeniyle güvenlik açısından önemli avantajlar sunduğunu ifade etmektedir.

Bu çalışmada geliştirilen sistem, literatürde belirtilen güvenlik, modülerlik ve erişim kontrolü problemlerine çözüm sunmayı amaçlamaktadır. Özellikle JWT tabanlı kimlik doğrulama, middleware yapısı ve rol bazlı yetkilendirme mekanizmaları sistemin güvenliğini artırmaktadır.

3. Yöntem

3.1 Sistem Mimarisi

Geliştirilen sistem istemci-sunucu (client-server) mimarisi kullanılarak tasarlanmıştır. Frontend tarafında React.js kullanılmış, backend tarafında ise Node.js ve Express.js tercih edilmiştir. Veritabanı işlemleri MySQL üzerinde gerçekleştirilmiş ve Sequelize ORM aracılığıyla yönetilmiştir.

Sistem MVC (Model-View-Controller) mimarisine uygun şekilde yapılandırılmıştır:

Model Katmanı: Veritabanı tablolarını ve ilişkilerini yönetmektedir.

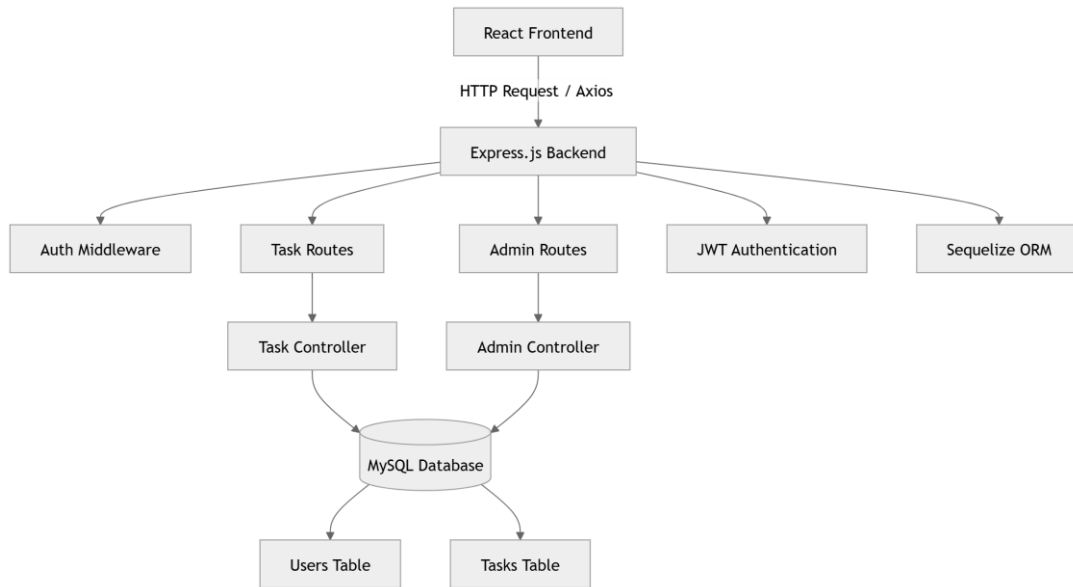
Controller Katmanı: İş mantığını ve veri akışını kontrol etmektedir.

Route Katmanı: API endpoint yapılarını yönetmektedir.

Middleware Katmanı: Kimlik doğrulama ve yetkilendirme işlemlerini gerçekleştirmektedir.

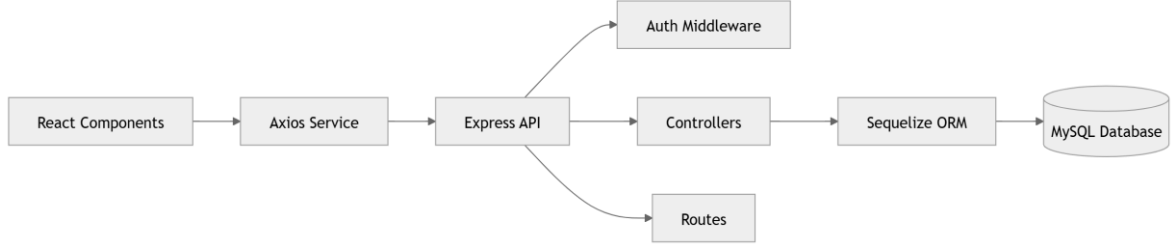
Frontend ve backend katmanlarının ayrılması sayesinde sistem daha sürdürülebilir ve geliştirilebilir hale gelmiştir.

Şekil 1: Sistem Mimarisi Diyagramı



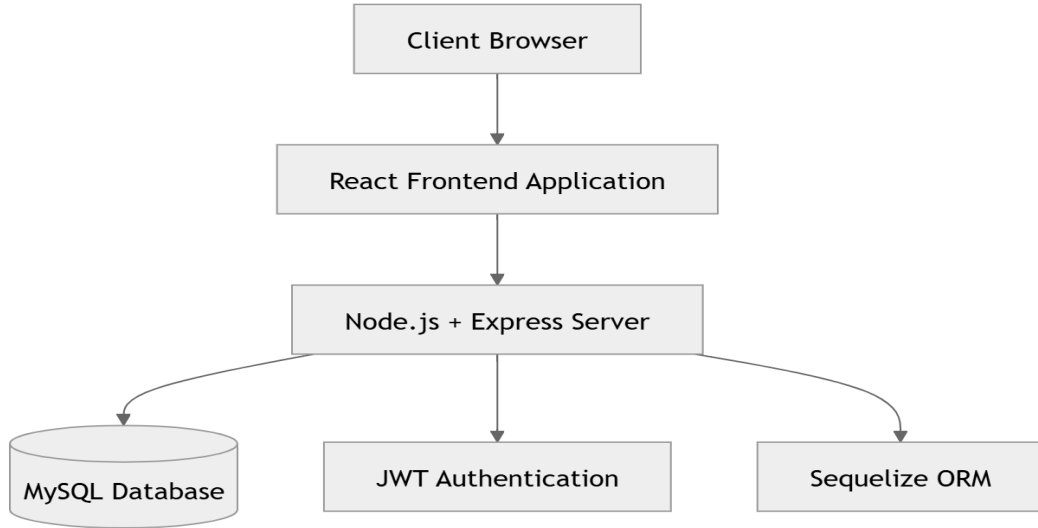
Bu diyagram sistemin frontend, backend, middleware ve veritabanı katmanları arasındaki ilişkiyi göstermektedir.

Şekil 2: Bileşen Diyagramı (Component Diagram)



Bu diyagram sistem bileşenlerinin birbirleriyle olan iletişimini göstermektedir.

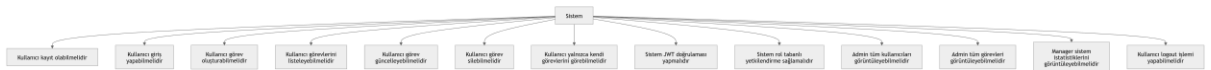
Şekil 3: Dağıtım Diyagramı (Deployment Diagram)



Bu diyagram sistemin dağıtım mimarisini ve istemci-sunucu yapısını göstermektedir.

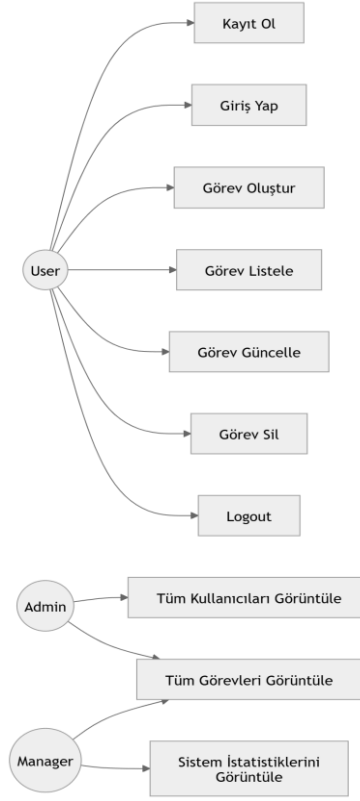
3.2 Fonksiyonel Gereksinimler

Şekil 4: Fonksiyonel Gereksinimler Diyagramı



Bu diyagram sistemin temel fonksiyonel gereksinimlerini göstermektedir.

Şekil 5: Kullanım Senaryosu Diyagramı (Use Case Diagram)



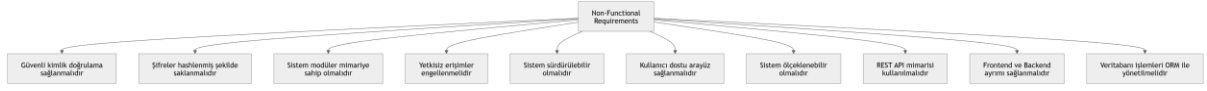
Bu diyagram sistem kullanıcılarının gerçekleştirebildiği işlemleri göstermektedir.

Sistem aşağıdaki temel fonksiyonel gereksinimleri karşılayacak şekilde geliştirilmiştir:

- FR1 → Kullanıcı kayıt olabilmelidir.
- FR2 → Kullanıcı giriş yapabilmelidir.
- FR3 → Kullanıcı görev oluşturabilmelidir.
- FR4 → Kullanıcı görevlerini görüntüleyebilmelidir.
- FR5 → Kullanıcı görev güncelleyebilmelidir.
- FR6 → Kullanıcı görev silebilmelidir.
- FR7 → Kullanıcı yalnızca kendi görevlerine erişebilmelidir.
- FR8 → Sistem JWT doğrulaması yapmalıdır.
- FR9 → Sistem rol tabanlı yetkilendirme sağlamalıdır.
- FR10 → Admin tüm kullanıcıları görüntüleyebilmelidir.
- FR11 → Admin tüm görevleri görüntüleyebilmelidir.
- FR12 → Manager sistem istatistiklerini görüntüleyebilmelidir.
- FR13 → Kullanıcı güvenli şekilde oturum kapatabilmelidir.

3.3 İşlevsel Olmayan Gereksinimler

Şekil 6: İşlevsel Olmayan Gereksinimler Diyagramı



Bu diyagram sistemin güvenlik, sürdürülebilirlik ve performans gibi işlevsel olmayan gereksinimlerini göstermektedir.

Sistemin işlevsel olmayan gereksinimleri aşağıdaki şekilde belirlenmiştir:

- NFR1 → Sistem güvenli kimlik doğrulama sağlamalıdır.
- NFR2 → Şifreler hashlenmiş şekilde saklanmalıdır.
- NFR3 → Sistem modüler mimariye sahip olmalıdır.
- NFR4 → Yetkisiz erişimler engellenmelidir.
- NFR5 → Sistem sürdürülebilir olmalıdır.
- NFR6 → Kullanıcı arayüzü kullanıcı dostu olmalıdır.
- NFR7 → Sistem ölçeklenebilir olmalıdır.
- NFR8 → Sistem REST mimarisine uygun olmalıdır.
- NFR9 → API endpoint yapıları güvenli olmalıdır.
- NFR10 → Veritabanı işlemleri yönetilebilir yapıda olmalıdır.

3.4 Kullanılan Teknolojiler

Bu çalışmada aşağıdaki teknolojiler kullanılmıştır:

- React.js
- Vite
- Node.js
- Express.js
- Sequelize ORM
- MySQL
- JWT (JSON Web Token)
- bcryptjs
- Axios
- Nodemon
- MySQL Workbench

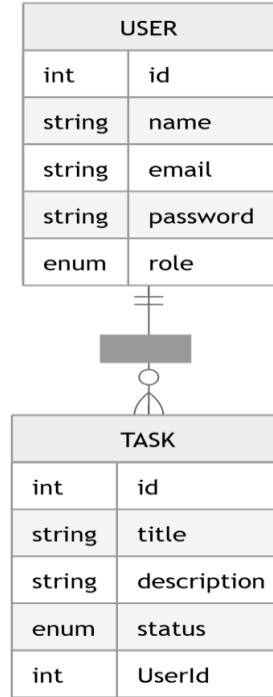
React.js frontend tarafında bileşen tabanlı kullanıcı arayüzü geliştirmek amacıyla kullanılmıştır. Vite, hızlı geliştirme ortamı ve optimize build süreci sağladığı için tercih edilmiştir.

Backend tarafında Express.js ile RESTful API mimarisi uygulanmıştır. Sequelize ORM, SQL sorgularını soyutlayarak veritabanı yönetimini kolaylaştırmıştır.

Axios kütüphanesi frontend ve backend arasında HTTP isteklerini gerçekleştirmek amacıyla kullanılmıştır. JWT kullanıcı kimlik doğrulama işlemlerinde tercih edilirken bcryptjs kullanıcı parolalarının güvenli şekilde hashlenmesini sağlamıştır.

3.5 Veritabanı Tasarımı

Şekil 7: ER Diagram



Bu diyagram Users ve Tasks tabloları arasındaki ilişkiyi göstermektedir.

Sistem temel olarak Users ve Tasks tabloları üzerinden çalışmaktadır. Users tablosunda kullanıcı bilgileri ve rol verileri tutulurken, Tasks tablosunda görev bilgileri saklanmaktadır.

Sistem içerisinde admin, manager ve user olmak üzere üç farklı kullanıcı rolü bulunmaktadır. Görev durumları ise todo, in-progress ve done değerlerinden oluşmaktadır.

Users ve Tasks tabloları arasında one-to-many ilişkisi kurulmuştur. Bir kullanıcı birden fazla göreve sahip olabilir.

3.6 Kimlik Doğrulama ve Güvenlik

Sistemde kullanıcı kimlik doğrulama işlemleri JWT kullanılarak gerçekleştirilmiştir. Kullanıcı giriş yaptıktan sonra sunucu tarafından token üretilmekte ve kullanıcı işlemlerinde Authorization header üzerinden doğrulama yapılmaktadır.

JWT doğrulama işlemleri middleware katmanında gerçekleştirilmektedir. Geçersiz veya süresi dolmuş token'lar sistem tarafından reddedilmektedir.

Şifre güvenliği için bcrypt algoritması kullanılmıştır. Kullanıcı şifreleri düz metin olarak saklanmamakta, hashlenmiş şekilde veritabanında tutulmaktadır.

Ayrıca sistemde rol tabanlı erişim kontrolü uygulanmıştır:

- Admin → Tüm kullanıcıları ve görevleri görüntüleyebilir.
- Manager → Sistem istatistiklerini görüntüleyebilir.
- User → Yalnızca kendi görevlerini yönetebilir.

Bu yapı sayesinde sistem güvenliği önemli ölçüde artırılmıştır.

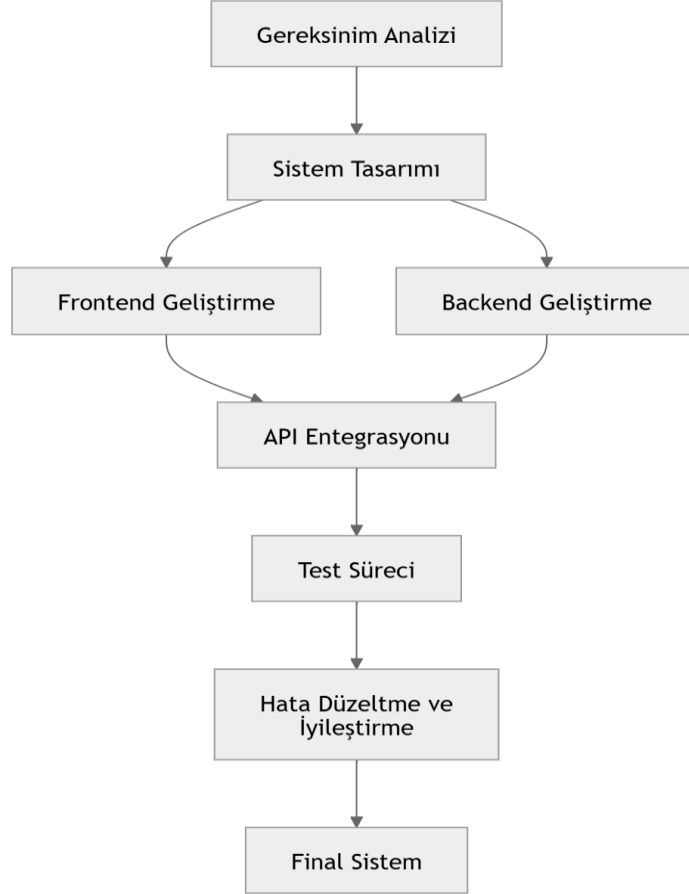
3.7 Yazılım Geliştirme Süreci

Bu projede çevik yazılım geliştirme yaklaşımı (Agile Development) kullanılmıştır. Proje geliştirme süreci analiz, tasarım, geliştirme, test ve iyileştirme aşamalarına bölünerek ilerletilmiştir.

İlk aşamada sistem gereksinimleri belirlenmiş ve kullanıcı ihtiyaçları analiz edilmiştir. Daha sonra frontend ve backend mimarileri tasarlanmıştır. Geliştirme sürecinde React tabanlı kullanıcı arayüzü ile Node.js ve Express.js tabanlı backend sistemi paralel şekilde geliştirilmiştir.

Her geliştirme aşamasından sonra sistem test edilerek hatalar düzeltilmiş ve güvenlik kontrolleri gerçekleştirilmiştir. Bu yaklaşım sayesinde sistem modüler, sürdürülebilir ve geliştirilebilir bir yapıda oluşturulmuştur.

Şekil 8: Yazılım Geliştirme Süreci Diyagramı



Bu diyagram proje geliştirme sürecinde izlenen temel yazılım mühendisliği aşamalarını göstermektedir.

4. Bulgular

Geliştirilen sistem farklı kullanıcı senaryoları üzerinde test edilmiştir. Test sonuçlarına göre sistemin temel işlevleri başarılı şekilde çalışmaktadır.

Elde edilen sonuçlar aşağıda sunulmaktadır:

- Kullanıcı kayıt işlemleri başarılı şekilde gerçekleştirilmektedir.
- JWT tabanlı giriş sistemi sorunsuz çalışmaktadır.
- Kullanıcılar görev oluşturabilmektedir.
- Görev listeleme işlemleri doğru şekilde çalışmaktadır.
- Görev silme işlemleri yetki kontrolü ile gerçekleştirilmektedir.

- Yetkisiz kullanıcıların admin endpoint'lerine erişimi engellenmektedir.
- Manager paneli sistem istatistiklerini doğru şekilde göstermektedir.
- Admin paneli kullanıcı ve görev yönetimini başarılı şekilde gerçekleştirmektedir.
- React tabanlı kullanıcı arayüzü dinamik veri güncellemelerini desteklemektedir.
- Sequelize ORM sayesinde veritabanı işlemleri düzenli şekilde yönetilmektedir.

Ayrıca sistem testleri sırasında aşağıdaki güvenlik kontrolleri gerçekleştirilmiştir:

- Geçersiz token kontrolü
- Yetkisiz erişim kontrolü
- Kullanıcı bazlı veri filtreleme
- Şifre hash doğrulama işlemleri

Test sonuçları sistemin hem işlevsel hem de güvenlik açısından başarılı olduğunu göstermektedir.

5. Tartışma

Bu çalışmada geliştirilen sistem modern yazılım mühendisliği prensiplerine uygun şekilde tasarlanmıştır. Sistem modüler mimariye sahip olduğu için bakım ve geliştirme süreçleri kolaylaştırılmıştır.

Frontend ve backend katmanlarının ayrılması yazılımın sürdürülebilirliğini artırmaktadır. Ayrıca REST API yapısı sayesinde sistem farklı platformlarla kolay şekilde entegre edilebilir hale gelmiştir.

JWT tabanlı kimlik doğrulama yapısı sayesinde sistem stateless mimariye uygun hale getirilmiştir. Bu yaklaşım performans açısından avantaj sağlamaktadır. Bununla birlikte JWT kullanımında token süresi ve token güvenliği dikkat edilmesi gereken önemli konular arasında yer almaktadır.

Rol tabanlı erişim kontrolü mekanizması sistem güvenliğini artırmıştır. Özellikle kullanıcıların yalnızca kendi görevlerine erişebilmesi veri gizliliği açısından önemli avantaj sağlamaktadır.

Sistemde Sequelize ORM kullanılması veritabanı işlemlerini daha okunabilir ve yönetilebilir hale getirmiştir. Ancak büyük ölçekli sistemlerde performans optimizasyonları ve cache mekanizmaları gerekebilir.

Benzer çalışmalar incelendiğinde birçok görev yönetim sisteminin yalnızca temel CRUD işlemlerine odaklandığı görülmektedir. Bu projede ise CRUD işlemlerinin yanı sıra rol tabanlı erişim kontrolü, JWT güvenliği, middleware yapısı ve yönetim panelleri de sisteme dahil edilmiştir.

Ayrıca React tabanlı dinamik kullanıcı arayüzü sayesinde kullanıcı deneyimi artırılmıştır. Kullanıcıların rollerine göre farklı dashboard ekranlarına yönlendirilmesi sistem kullanımını daha profesyonel hale getirmiştir.

6. Sonuç

Bu çalışmada React, Node.js, Express.js, Sequelize ORM ve MySQL kullanılarak güvenli, modüler ve rol tabanlı erişim kontrolüne sahip bir görev yönetim sistemi geliştirilmiştir.

Sistem modern yazılım mühendisliği prensipleri doğrultusunda analiz edilmiş, tasarlanmış ve uygulanmıştır. JWT tabanlı kimlik doğrulama, bcrypt ile parola güvenliği ve middleware tabanlı yetkilendirme yapıları sayesinde güvenli bir sistem oluşturulmuştur.

Frontend ve backend katmanlarının birbirinden ayrılması sistemin sürdürülebilirliğini artırmıştır. Ayrıca sistem ölçeklenebilir yapıda geliştirildiği için gelecekte yeni özelliklerin eklenmesine uygun hale getirilmiştir.

Bu proje sayesinde aşağıdaki kazanımlar elde edilmiştir:

- RESTful API geliştirme deneyimi
- React tabanlı frontend geliştirme deneyimi
- JWT ile güvenli kimlik doğrulama
- Sequelize ORM kullanımı
- Middleware mimarisi
- Rol tabanlı erişim kontrolü
- Modern yazılım mimarisi prensipleri

Gelecekte yapılabilecek geliştirmeler aşağıdaki gibidir:

- Görev önceliklendirme sistemi
- Docker entegrasyonu
- Bulut tabanlı deployment
- Mobil uygulama desteği
- Gerçek zamanlı görev güncellemeleri
- Redis cache desteği
- API rate limiting mekanizması
- E-posta doğrulama sistemi

Sonuç olarak geliştirilen sistem, hem akademik hem de teknik açıdan başarılı bir full-stack web uygulaması örneği sunmaktadır.

Kaynakça

- Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California.
- Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT). Internet Engineering Task Force.
- Pressman, R. S. (2014). Software Engineering: A Practitioner's Approach. McGraw-Hill.
- Stallings, W. (2017). Cryptography and Network Security: Principles and Practice. Pearson.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- Pautasso, C., Zimmermann, O., & Leymann, F. (2008). RESTful Web Services vs. Big Web Services. WWW Conference.
- Zhang, Q., Chen, M., & Li, L. (2019). Design and Implementation of Task Management System Based on Web. International Journal of Computer Applications.